FOOD WASTE REDUCTION CUM MANAGEMENT SYSTEM

An AI-Driven Approach to Demand Forecasting, Dynamic Pricing, and Equitable Surplus Redistribution in Institutional Cafeterias

PHASE – 1 REPORT

SUBMITTED BY

JAYAPRAKASH A 230701129

KAMALESH P 230701137

KARTHIKA LAKSHMI V 230701142

MITHUL AADITYA A 230701185

in partial fulfilment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY, CHENNAI 600 025

September 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled **"FOOD WASTE REDUCTION CUM MANAGEMENT SYSTEM: AN AI-DRIVEN APPROACH TO DEMAND FORECASTING, DYNAMIC PRICING, AND EQUITABLE SURPLUS REDISTRIBUTION IN INSTITUTIONAL CAFETERIAS"** is the bonafide work of **JAYAPRAKASH A (230701129), KAMALESH P (230701137), KARTHIKA LAKSHMI V (230701142)** and **MITHUL AADITYA A (230701185)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

SIGNATURE

Dr. J. Manoranjini
Professor & Head
Department of Computer Science and Engineering,
Rajalakshmi Engineering College, Thandalam, Chennai - 602105.

SIGNATURE

Ms. M. Bhavani
Assistant Professor
Department of Computer Science and Engineering,
Rajalakshmi Engineering College, Thandalam, Chennai - 602105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner **External Examiner**

ABSTRACT

Cafeterias attached to corporate campuses, colleges, and schools still largely run on guesswork: one menu, one price list, and preparation quantities set by whoever is on kitchen duty that morning, regardless of whether the crowd that day is a price-sensitive school canteen or a premium corporate one. That mismatch shows up as food thrown away at close of service and as surplus that, even when it exists, usually goes to whichever NGO happened to be called first. This project, the Food Waste Reduction cum Management System, was built to close that gap, aligned with UN Sustainable Development Goal 12 (Responsible Consumption and Production).

The system couples seven cooperating decision engines around a single FastAPI backend: an XGBoost model that forecasts daily demand per dish, a closed-form pricing rule that adjusts price within a cost-recovery floor and a segment-specific ceiling, a Random Forest model that predicts end-of-day leftovers, a transparent scoring function that recommends tomorrow's menu, a Gradient Boosting classifier that targets discount offers at customers likely to convert, and a constraint-based CP-SAT optimizer that splits surplus food fairly across NGO partners. A supervisor process runs all seven engines in sequence once a day as an audited pipeline, logging every decision made.

Because no public dataset records the combination of unit cost, quantity prepared, and institutional segment this system needs, a documented synthetic data generator was built, and the three trained models were evaluated against a strict, time-ordered holdout rather than a random split. On the generated dataset, the forecasting model reaches an R^2 of 0.875 (a 45.0% error reduction over a naive weekly-average baseline), the waste model reaches an R^2 of 0.839, and the conversion model achieves an AUC of 0.658 with a 2.03× lift over blanket discounting. The allocation engine keeps every NGO partner above its guaranteed 20% share while covering 99.4% of available surplus, and the pricing floor was never once breached across all recorded runs.

By combining demand-driven preparation planning, segment-fair dynamic pricing, targeted trial-conversion offers, and equitable surplus redistribution in one auditable platform, the system aims to reduce operational waste and improve margin for institutional cafeterias while giving verified NGO partners a fair, guaranteed, and predictable share of surplus food.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E., F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavouring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. J. MANORANJINI, Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our Project Guide, Ms. M. BHAVANI, Assistant Professor, Department of Computer Science and Engineering, for her valuable guidance throughout both the survey and implementation phases of this project.

We also extend our sincere thanks to all the faculty members of the Department of Computer Science and Engineering, Rajalakshmi Engineering College, for their support and encouragement. Finally, we express our heartfelt gratitude to our parents, friends, and well wishers for their constant motivation and support throughout the successful completion of this project.

TABLE OF CONTENTS

Section	Title	Page No.
CHAPTER 1	**INTRODUCTION**	**9**
1.1	General	9
1.2	Objective	10
1.3	Existing System	11
1.4	Proposed System	11
CHAPTER 2	**LITERATURE SURVEY**	**13**
2.1	Literature Review	13
2.2	Comparison and Discussion	14
2.3	Conclusion	15
CHAPTER 3	**SYSTEM DESIGN**	**16**
3.1	System Architecture	16
3.2	System Requirements	17
3.2.1	Software Requirements	17
3.2.2	Hardware Requirements	18
3.2.3	Dataset Requirements	18
3.2.4	Deployment and Scaling	19
3.3	System Design	19
3.3.1	Agent Pipeline Diagram	20
3.3.2	Data Flow Diagram	20

CHAPTER 4	**PROJECT DESCRIPTION**	**22**
4.1	Methodologies	22
4.2	Modules	22
4.2.1	Data and Auth Module	23
4.2.2	Pipeline module	24
4.2.3	Pricing and floor enforcement module	24
4.2.4	Menu and Trial Conversion Module	25
4.2.5	NGO Surplus Distribution Module	25
4.2.6	UI / DashBoard Module	26
4.3	Seven Engines	26
CHAPTER 5	**IMPLEMENTATION AND RESULT DISCUSSION**	**28**
5.1	Implementation Results	28
5.1.1	Implementation of Data Set Generation	29
5.1.2	Implementation of Model Training and Evaluation	30
5.1.3	Implementation of Daily Pipeline	31
5.2	Discussion	32
CHAPTER 6	**CONCLUSION AND FUTURE WORK**	**34**
6.1	Conclusion	34
6.2	Future Work	34
	REFERENCES	35

LIST OF TABLES

Table No.	Title	Page No.
Table 1	Comparative Analysis	14
Table 2	Software Requirements	17
Table 3	Hardware Requirements	18
Table 4	The Seven Engines and What Drives Each One	27
Table 5	Summary of System Performance Metrics	31

LIST OF FIGURES

Figure No.	Title	Page No.
Fig. 1	Overview of Proposed System	12
Fig. 2	Architecture Diagram	16
Fig. 3	Agent Pipeline Diagram	20
Fig. 4	Data Flow Diagram	21
Fig. 5	Demand Forecasting	28
Fig. 6	UI DashBoard	29
Fig. 7	Comparative Bar Graph	32

LIST OF ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AUC	Area Under the (ROC) Curve
CP-SAT	Constraint Programming — Satisfiability
CRUD	Create, Read, Update, Delete
DFD	Data Flow Diagram

ER	Entity-Relationship
FK	Foreign Key

GB	Gradient Boosting
IC	Institution Coordinator
JWT	JSON Web Token
KM	Kitchen Manager
MAE	Mean Absolute Error
MAPE	Mean Absolute Percentage Error
ML	Machine Learning
MVP	Minimum Viable Product
NGO	Non-Governmental Organization
ORM	Object-Relational Mapping
PK	Primary Key
RBAC	Role-Based Access Control
REST	Representational State Transfer
R^2	Coefficient of Determination
RF	Random Forest
SDG	Sustainable Development Goal
SQL	Structured Query Language
UN	United Nations
XGB	Extreme Gradient Boosting

CHAPTER 1

INTRODUCTION

1.1 General

Cafeterias attached to corporate campuses, colleges, and schools still largely run on guesswork: one menu, one price list, and preparation quantities set by whoever is on kitchen duty that morning, regardless of whether the crowd that day is a price-sensitive school canteen or a premium corporate one. As footfall and menu variety increase, this manual planning model struggles to keep pace, leading to over-preparation, food wastage, lost margin, and surplus food that is inconsistently — if ever — routed to organizations that could use it.

The challenge becomes clear when cafeteria operations must manage changing customer volumes and varying consumption patterns day by day. A dish that sells one day may be in short supply another day so keeping a fixed amount ready can lead to shortages or excess. Pricing decisions face an issue because customers, in different institutional segments may react differently to the same price or discount. At the end of the day the remaining food must be identified quickly. Matched to suitable organizations before it becomes useless. Without a system these tasks stay separated with demand planning, pricing, waste monitoring, promotional offers and surplus distribution handled on their own. Bringing these decisions together helps cafeteria teams respond to changes more systematically cuts unnecessary waste and improves the use of food that would otherwise stay unused.

The advent of applied machine learning and constrained optimisation presents a transformative opportunity to automate and rationalise these decisions. The Food Waste Reduction cum Management System leverages these techniques to build an intelligent decision-support layer over the day-to-day operations of an institutional cafeteria — forecasting demand, adjusting price within safe bounds, predicting leftovers before they occur, recommending tomorrow's menu, targeting promotional offers only at customers likely to respond, and distributing genuine surplus fairly among verified NGO partners.

The project is explicitly aligned with UN Sustainable Development Goal 12 (Responsible Consumption and Production), and is scoped for three institutional segments that behave very differently in practice — a corporate tech-park cafeteria, a college, and a school — rather than treating institutional food service as a single undifferentiated market.

1.2 Objective

The first objective of the system is to reduce food wastage by replacing manual preparation estimates with data-driven demand forecasting. Of relying only on the kitchen teams experience or previous days sales the system predicts the expected demand for each dish at each institution. This helps kitchen managers plan preparation quantities carefully and respond to differences in customer volume and eating patterns. By producing a forecast for different dishes and institutions the system can account for the fact that demand is not the same everywhere. The overall aim is to prepare to what customers are actually expected to consume while still maintaining sufficient availability during the day.

The second objective is to improve cafeteria profitability through a pricing mechanism that responds to changing demand and waste conditions. The system considers factors such as expected demand, potential waste pressure and customer segment before generating a recommended price. At the time the pricing logic applies a strict cost-recovery floor so that automated adjustments cannot reduce the selling price below the defined minimum. This allows pricing to remain flexible without ignoring the requirement of recovering costs. The approach is intended to provide kitchen managers with a pricing recommendation rather than requiring them to repeatedly estimate suitable prices manually.

The third objective is to improve customer engagement and recover revenue through targeted trial-conversion offers. Than giving discounts to every customer the system identifies customers who have not purchased a selected dish and estimates their likelihood of responding to an offer. The conversion model helps prioritize customers where a limited promotional budget may have practical value. Discounts are also adjusted according to estimated price sensitivity. Remain subject to the systems pricing safeguards. This creates a controlled approach to promotions, where offers are generated from customer behaviour and reviewed before being delivered.

The fourth objective is to make the redistribution of food surplus more organized, fair and practical. Once the system identifies the leftover quantity the surplus is matched with verified NGO partners based on factors such as dietary compatibility and receiving capacity. The allocation process is designed to prevent the system from assigning food than is actually available or more than a partner can reasonably accept. A minimum-share rule is also applied to partners selected for the days distribution helping maintain an allocation while respecting individual capacity limits. In this way usable surplus can move through a defined process of depending on last-minute manual coordination.

The final objective is to provide transparency and accountability across the automated workflow. Super Admins and Institution Coordinators should be able to understand what decisions were generated by the system and review the operational information without having to inspect the underlying code or database manually. Each agents activity can be recorded with details such, as its action, execution status, processing information and completion time. The objective is therefore not to automate cafeteria operations but also to

make the resulting decisions understandable traceable and easier to monitor.

1.3 Existing System

Current institutional cafeteria operations in corporate, college, and school settings rely on manual, experience-based decision-making at nearly every stage:

- Preparation quantities are estimated from memory and habit, with no historical-data support and no adjustment for weather, calendar effects, or exam periods.
- Menu pricing is fixed or only occasionally revised, and rarely responds to real-time demand, cost changes, or waste risk.
- Inventory management is reactive — stock is reordered only after shortages or spoilage are already visible.
- Personalization is minimal to non-existent: the same menu and the same price are shown to every customer regardless of segment or purchase history.
- Surplus food generated at day-end is handled inconsistently, and is frequently discarded rather than redistributed, because there is no structured process connecting the kitchen to NGO partners.
- Sales, inventory, and feedback data remain siloed across paper logs or disconnected spreadsheets, preventing any integrated, data-driven decision-making across the cafeteria's operations.

1.4 Proposed System

The proposed Food Waste Reduction cum Management System replaces this guesswork with seven cooperating decision engines, run once a day as an audited pipeline: a demand forecasting engine (XGBoost) that predicts how much of each dish will sell at each institution; a dynamic pricing engine that computes a recommended price using a closed-form, constant-elasticity economic model bounded by a hard cost-recovery floor and a segment-specific ceiling; a waste/surplus prediction engine (Random Forest) that estimates end-of-day leftovers from the kitchen's own preparation decision; a menu recommendation engine that scores candidate dishes on a transparent, explainable, multi-factor basis; a trial-conversion engine (Gradient Boosting) that targets discount offers only at non-buying customers likely to respond; and an NGO surplus allocation engine (Google OR-Tools CP-SAT) that splits genuine surplus fairly across verified NGO partners subject to dietary compatibility, partner capacity, and a guaranteed minimum share.

These engines work together as a decision−making workflow instead of operating as separate prediction models. The forecasting engine output gives the expected demand that helps with

preparation and pricing decisions. The waste engine uses the planned preparation quantity to estimate the surplus. The menu engine then helps find dishes by looking at demand and other operational aspects.

The trial−conversion engine uses customer signals to find where a small promotional budget can be used effectively. Any surplus that remains after normal sales goes to the NGO allocation engine. That engine looks at partner requirements and capacity before making a distribution plan. Each stage records an output that can be reviewed through the system's audit trail. This makes the overall process easier to understand, monitor and improve while keeping operational decisions, inside defined business rules.

Deliberately, not every decision is handed to a learned model. Pricing needs a hard, human-auditable floor a kitchen manager can trust and approve every day, and NGO allocation needs constraints — dietary compatibility, partner capacity, a fairness guarantee — that must hold exactly rather than on average. A model can only be hoped to respect a constraint like that; a constraint solver enforces it exactly. Machine learning is used only where the underlying decision genuinely benefits from statistical generalisation over noisy, continuous, real-world data: demand, leftover quantity, and conversion probability.

For Phase 1, the system's scope is deliberately bounded: demand forecasting, approval-gated pricing, inventory tracking, NGO allocation, core analytics, and weather-based decision support are included and functional; billing workflows, an integrated payment gateway, and deeper personalization are treated as future work outside the current phase. A kitchen manager remains firmly in the loop — every recommended price, menu, and promotional offer is reviewed and approved before it takes effect, and every kitchen-manager approval or override is fed back into the pricing engine as a ground-truth correction signal.

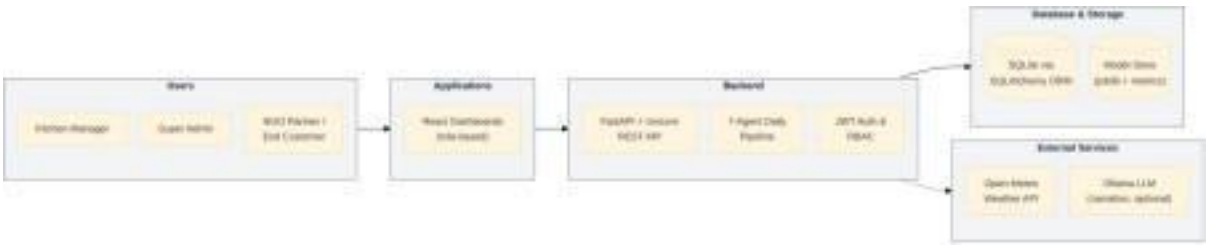


Fig. 1 — Overview of Proposed System

As illustrated in Fig. 1, the proposed system connects Kitchen Manager, Super Admin, NGO Partner, and End Customer users through role-based React dashboards to a central FastAPI backend. This backend executes the seven-agent daily decision pipeline and JWT-based authentication, while persisting data to a SQLite database and a joblib model store, and integrating with the Open-Meteo weather API and an optional local Ollama LLM for narration

CHAPTER 2

LITERATURE SURVEY

2.1 Literature Review

The landscape of AI-driven food-service management has experienced a steady evolution from purely manual, experience-based kitchen planning toward data-driven, model-assisted decision-making across forecasting, personalisation, waste prevention, and donation logistics. Early and ongoing research on institutional demand forecasting has applied machine learning directly to real cafeteria and refectory demand histories. Aydın et al. (2025) evaluated a range of ML techniques on an institutional cafeteria's own demand history, establishing a practical baseline for the problem, and Aci and Yergök (2023) reached a similar conclusion using a university refectory's demand records. Agarwal et al. found that blending classical statistical methods with machine learning outperformed either approach on its own for food-demand prediction, while Hübner et al. (2024) went a step further and connected demand forecasting directly to environmental outcomes, showing in a bakery case study that better forecasts reduce the life-cycle impact of over-production. Posch et al. (2022) took a Bayesian route instead of point forecasts, useful precisely when a canteen's sales history is too short for a confident regression estimate, and Anil Kumar et al. (2025) combined gradient-boosted ensembles (XGBoost and CatBoost together) with statistical features tied to location and customer satisfaction. Taken as a group, these studies confirm that demand forecasting for food service is a mature and well-studied problem, but none of them distinguish between institution types the way a corporate cafeteria, a college, and a school actually differ — the distinction this project's own forecasting engine is built around.

A second line of research asks a different question: not how much food will be sold in aggregate, but what a specific person is likely to want. Ajami and Teimourpour (2023) built a hybrid recommender for an academic dining hall that combined collaborative filtering, content-based matching, and knowledge rules, and found an AdaBoost-based hybrid outperformed simpler baselines on a dataset of over 2,500 students. Ju et al.'s MenuAI showed that transformer architectures developed for language tasks transfer reasonably well to restaurant menu recommendation, and Rokon et al. (2022) took the problem down to the ingredient level, pairing a CNN-based ingredient detector with a recipe-matching algorithm at 94% classification accuracy. These systems are strong at predicting individual taste but say little about how a menu or a price should shift with the institutional segment being served, which is the gap this project's menu-recommendation and pricing engines are meant to fill.

A third cluster targets waste directly, by changing what gets ordered, cooked, or offered before any of it becomes surplus. Aziz et al. (2026) framed menu selection itself as a fair-allocation problem, choosing a menu across people with different dietary needs and appetite sizes so as to minimise wastage under both optimistic and pessimistic assumptions about how much people actually eat. Nu, Belavina, and Girotra (2024) measured, empirically, how much predictive and inventory-aware intervention actually reduces discarded food in restaurant operations, and both Ozcil (2024) and Yang, Zhao, and Lu (2024) looked specifically at how well different ML models predict and reduce food waste. This cluster is about stopping waste on the supply side — exactly what this project's waste-prediction engine is built to inform, by feeding its leftover estimate back into both the kitchen's preparation quantity and the pricing markdown.

The largest and most methodologically diverse cluster addresses what happens once food does become surplus. Dalal (2022) modelled food-donation planning under COVID-era supply and demand uncertainty as a robust optimisation problem, while Sundin et al. (2023) assessed the sustainability of the donation pathway itself, treating it as a transfer system with measurable environmental, economic, and social value. A ScienceDirect (2024) study integrates demand forecasting directly into a multi warehouse allocation model, and a 2025 study in the Journal of Humanitarian Logistics extends the same idea into vehicle routing, jointly optimising delivery efficiency and fairness among partner agencies. This is a genuinely deep literature, but nearly all of it is built for city-scale food banks serving many, often anonymous, recipients — not for an institutional cafeteria distributing surplus among a small, known, capacity-constrained set of NGO partners, which is the smaller and more relationship

aware version this project's NGO allocation engine actually solves.

2.2 Comparison and Discussion

A rigorous comparative analysis of the reviewed literature reveals distinct trade-offs across aggregate demand forecasting, personalised recommendation, waste-prevention optimisation, and equitable surplus donation (Table 1).

Table 1 — Comparative Analysis

Evaluation Dimension	Aggregate Demand Forecasting	Personalised Food Recommendation	Waste-Prevention Optimisation	Equitable Surplus Donation
Core Techniques	XGBoost, CatBoost, Bayesian models, classical statistics	Collaborative filtering, transformers, CNN + recipe matching	Optimisation-aware menu selection, predictive + prescriptive ML	Robust optimisation, game theory, vehicle routing, fair allocation
Primary Advantages	Mature, well-studied problem; strong accuracy on	Captures individual taste; improves personalised engagement	Directly targets the point where waste is generated	Deep, methodologically diverse literature for city-scale

	institutional data			allocation
Critical Limitations	None distinguish institution types (corporate vs. college vs. school)	Says little about how price or menu should shift by segment	Rarely coordinated with pricing or donation decisions	Built for city-scale food banks with many anonymous recipients, not a small known partner set
Gap This Project Targets	Segment-differentiated forecasting feeding a cost-recovery-bounded price	Fold personalization into a targeted, elasticity aware discount engine	Feed waste prediction directly into the pricing markdown	A smaller, capacity and fairness constrained allocator for verified NGO partners

The discussion of these findings indicates that no single cluster above covers more than two of the four capabilities this system needs, and none of the reviewed sources addresses segment-aware pricing, targeted promotional conversion, an enforced cost-recovery floor, or coordination across multiple decision engines at once. Aggregate forecasting methods are accurate but institution-agnostic; personalised recommendation captures individual taste but not segment-level pricing; waste-prevention work rarely feeds its predictions into a pricing decision; and the donation-allocation literature, while methodologically the richest, is built for a different scale of problem — many anonymous recipients rather than a small, known, capacity-constrained NGO partner set. Consequently, current research highlights a clear gap for a coordinated, segment-aware system that closes the loop between forecasting, pricing, waste prediction, and fair surplus distribution within a single institutional cafeteria.

2.3 Conclusion

The comprehensive review of existing literature establishes a robust technical and theoretical foundation for designing a segment-aware, multi-engine cafeteria management system. By evaluating past methodologies — ranging from ensemble and Bayesian demand forecasting to transformer-based menu recommendation, waste-minimising menu selection, and robust or game-theoretic donation allocation — the survey provides actionable building blocks for resolving institutional food-service inefficiencies. Specifically, combining institution-aware demand forecasting with a closed-form, floor protected pricing rule, a waste-prediction feedback loop into that pricing decision, and a capacity- and fairness-constrained NGO allocator offers a coordinated pathway that no single reviewed source attempts on its own, which is the gap this project's own earlier survey identified and set out to close

CHAPTER 3

SYSTEM DESIGN

3.1 System Architecture

The system runs as a single FastAPI service on top of a SQLAlchemy ORM over SQLite, structured into four primary layers that ensure clear separation of concerns. The Client Layer comprises role-based React 19 / Vite / Tailwind CSS dashboards for the Super Admin, Kitchen Manager, Institution Coordinator, NGO Partner, and End Customer. The API Layer is a FastAPI + Uvicorn server exposing 40 REST endpoints, authenticating every request with a JWT bearer token scoped to one of the five roles. The Agent Pipeline Layer is a daily 7-agent orchestration pipeline, run in-process by a supervisor rather than as separate microservices, that produces the forecast, price, menu, offer, and NGO-allocation decisions. The Data & External Layer persists state to SQLite and a joblib model store, and integrates with the Open-Meteo weather API and an optional local Ollama LLM used only to narrate the daily summary.

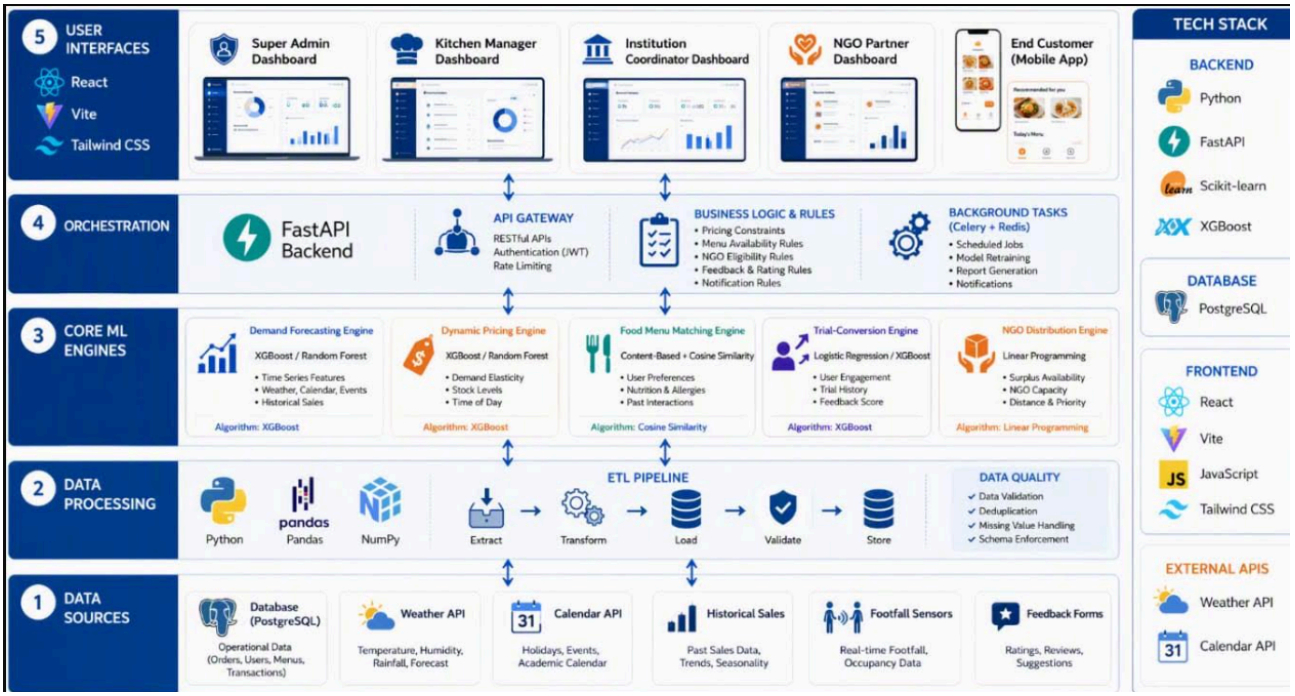


Fig. 2 — Architecture Diagram

As illustrated in Fig. 2, the system connects five role-based dashboards to a central FastAPI gateway, which triggers the seven-agent daily pipeline (Context, Forecast, Waste, Pricing, Menu, Offer, NGO, and a closing Supervisor step).

The pipeline reads from and writes to the SQLite data layer and the joblib model store, and reaches out to external services — Open-Meteo for live weather and, optionally, a local Ollama model for narration — without blocking the core decision logic if either is unavailable.

The system treats these external services as supporting components rather than making them a dependency for the main decision-making process. Weather information can be used as an additional context signal when generating demand-related recommendations, while the optional Ollama service can provide more natural explanations of selected system outputs. If the weather service is temporarily unavailable or the local language model is not running, the core pipeline can continue using the available database records and trained models

3.2 System Requirements

The execution and deployment of the Food Waste Reduction cum Management System require a modest, CPU-only software and hardware environment — a deliberate choice, since none of the system's seven engines require GPU acceleration.

3.2.1 Software Requirements

Table 2 — Software Requirements

Software Component	Technology / Tool	Version / Specification
Operating System	Ubuntu Linux / Windows (WSL2)	Ubuntu 22.04 LTS recommended
Backend Framework	FastAPI + Uvicorn	40-endpoint REST API, auto docs at /docs
ORM / Database	SQLAlchemy ORM over SQLite	19 tables; one connection-string change swaps in PostgreSQL
Authentication	JWT (python-jose) + passlib/pbkdf2	Role-based access control across 5 roles
ML Libraries	XGBoost, scikit-learn, pandas, numpy, joblib	As pinned in requirements.txt
Constrained Optimisation	Google OR-Tools CP-SAT	NGO surplus allocation engine
Orchestration	Hand-rolled supervisor + 7-agent pipeline	Python implementation, per-agent audit logging
Optional LLM	Ollama (local llama3.2)	Narration only; template fallback if not running
External API	Open-Meteo (free, keyless)	Live weather; offline synthetic fallback
Frontend	React 19, Vite, Tailwind CSS, react-router dom, Recharts	Role-based dashboards

3.2.2 Hardware Requirements

Table 3 — Hardware Requirements

Hardware Category	Specification	Purpose in System
Processor (CPU)	Multi-core x86_64 (Intel i5 / Ryzen 5 or higher)	Serves the FastAPI backend, ML inference, and CP-SAT solves
System Memory	Minimum 8 GB RAM (16 GB recommended)	Model training (XGBoost/Random Forest/Gradient Boosting) and pandas processing
Storage	Minimum 5 GB free	SQLite database, joblib model store, dataset generation output
GPU	Not required	All engines run on CPU; no deep-learning training in the current scope
Network	Standard broadband	Live weather API calls (Open-Meteo); degrades gracefully offline

3.2.3 Dataset Requirements

No public dataset records the combination of unit cost, quantity prepared, quantity leftover, institution segment, and NGO need profiles that the pricing-floor, waste, and NGO engines all depend on — UCI's Online Retail, the obvious public alternative, is UK gift-ware e-commerce and would raise an unanswerable “why does this generalise” question for a canteen forecaster. The project therefore relies on a documented synthetic data generator (Section 5.1.1) producing 365 days of aggregate daily sales (29,044 rows) across three institutions plus 45 days of individual, per-customer orders for roughly 500 synthetic customers, with every generative assumption fixed before training rather than tuned afterward.

The synthetic dataset was built to reflect the real-world conditions that the proposed system is expected to handle. Of trying to fit an unrelated public dataset into the problem we created data that matches what the system will actually face. The generator was set up to produce connections between demand how much food is prepared, leftover food, pricing, customer behavior details about the institution and the needs of NGOs. This ensures that all decision engines can work with the base of data. By locking in the assumptions before training the models we avoid adjusting how the data is generated to get better results. The dataset includes a range of variations across different institutions, types of food, customer groups and daily demand trends. This allows us to test the pipeline under many different situations. Because of this the dataset works well for development and controlled testing. Still it's important to remember that results from data cannot be taken as proof of performance, in an actual canteen setting.

3.2.4 Deployment and Scaling

The current implementation runs as a single FastAPI process suitable for a pilot-scale deployment at one institution; the three trained models are trained offline ahead of time, each producing a saved joblib model file and a JSON metrics sidecar, so the live daily pipeline only ever loads and predicts — it never retrains on a live request. This keeps the pipeline's per-run latency low (about three seconds for all seven agents) and keeps every day's decisions traceable back to a specific model version. Horizontal scaling to multiple institutions and a managed PostgreSQL instance is identified as future work once the system is validated against a real cafeteria's data (Section 6.2).

The system was designed to be simple so that it could be shown and tested without making things too complicated. The FastAPI service manages the real-time API requests. Makes sure the daily tasks for the agent run properly. The machine learning models are kept separate from the program and can be changed when a new approved version comes out. Because the system uses models that are already trained it does not waste time training models or changing settings while it is working. Every time a process runs it saves the model and the choices made so it is easy to see which version of the model was responsible, for a suggestion. If the system needs to be bigger the same setup can be expanded by splitting up the services using PostgreSQL for the database adding background tasks and allowing than one group to use it without changing the main way decisions are made.

3.3 System Design

The system design bridges the high-level architecture of Section 3.1 with the concrete decision flow that runs once a day, defining both the sequence in which the seven agents execute and how data moves between the system and its external actors.

The pipeline works like a chain of steps where each part uses the information from the step before it of making decisions on its own. The information gathered at the start sets the situation for the days predictions. The forecast that comes out of that then helps with planning. Figuring out how much waste there might be. The numbers, for waste and what people want then help decide prices and what to put on the menu. After that special tests and how to handle extra items happen. When all the steps are done the Supervisor puts everything together into a report and writes down what happened so it can be looked at again later. This way the system takes data and turns it into useful advice in a clear and organized way while keeping track of how each choice was made.

3.3.1 Agent Pipeline Diagram

The pipeline workflow models the end-to-end execution path of one daily run and the corresponding system reactions across its seven agents. Triggered by a “Run daily pipeline” action on the Kitchen Manager dashboard, a Context agent first gathers live weather and calendar data; a Forecast agent predicts demand and a newsvendor-adjusted preparation quantity per dish; a Waste agent uses that preparation quantity to predict leftover units; a Pricing agent turns the waste estimate and demand forecast into a recommended price, always subject to the cost-plus-20% floor; a Menu agent ranks tomorrow's candidate dishes; an Offer agent scores yesterday's non-buyers for conversion likelihood; and an NGO agent solves the fairness-constrained surplus allocation across partner NGOs. A closing Supervisor step assembles a summary, optionally narrates it, and writes an audit log row for every agent.

Each agent step is wrapped so that its own failure is caught and logged as an error rather than aborting the whole run.



Fig. 3 — Agent Pipeline Diagram

As illustrated in Fig. 3, the pipeline maps out the sequential execution flow of the system, detailing the operational logic from context gathering through demand forecasting, waste prediction, floor-protected pricing, menu ranking, and targeted offer generation, down to fairness-constrained NGO surplus allocation and final audit logging.

3.3.2 Data Flow Diagram

The data flow diagram governs how requests, ML predictions, and kitchen-manager approvals propagate across the system without introducing latency bottlenecks. At Level 0, external actors — Super Admin, Kitchen Manager, Institution Coordinator, NGO Partner, and End Customer — interact with the system as a single process; at Level 1, that process decomposes into the auth, ordering, forecasting, pricing, waste, menu, trial-conversion, NGO-allocation, and analytics processes shown below, each reading from and writing to its own data store.

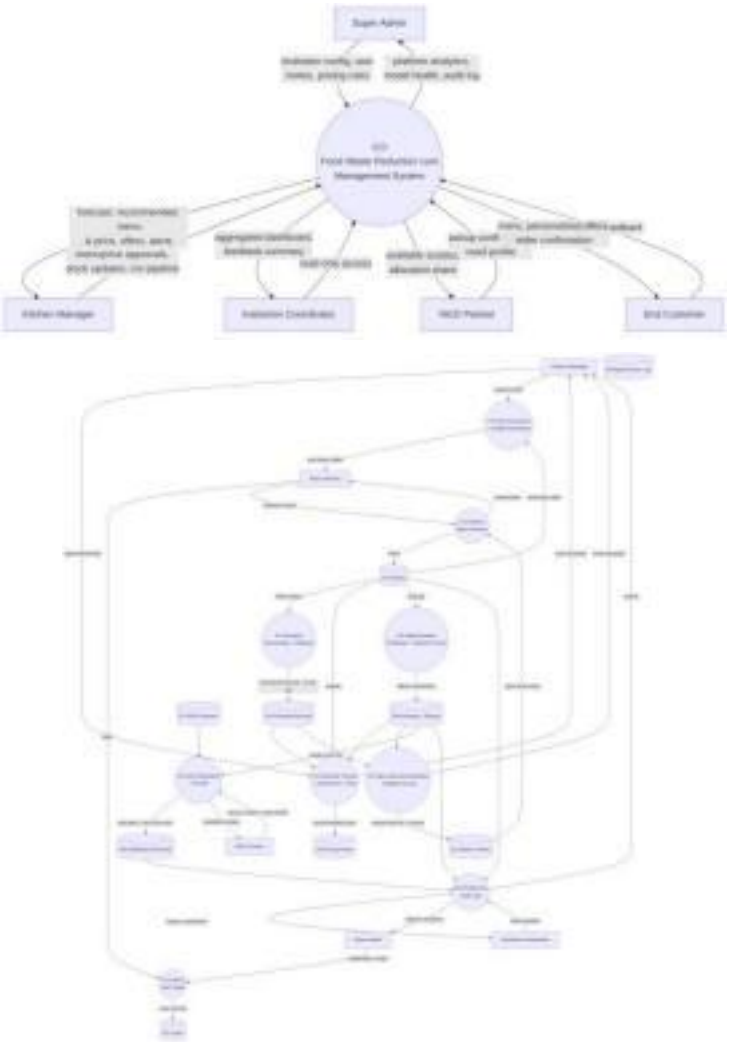


Fig. 4 — Data Flow Diagram (Level 0 Context and Level 1)

As illustrated in Fig. 4, the data flow diagram portrays the directional movement of information across the platform: the Super Admin and Kitchen Manager configure and operate the system, the End Customer and NGO Partner consume its output, and the Institution Coordinator reads aggregated data without ever writing into any engine. Kitchen Manager approvals and overrides on the recommended price notably feed back into the Pricing process as a ground-truth correction signal, closing the loop rather than leaving it a one-way recommendation.

CHAPTER 4

PROJECT DESCRIPTION

4.1 Methodologies

The Food Waste Reduction cum Management System implements a deliberately mixed methodological framework rather than applying one technique everywhere, coordinated by a daily supervisor-driven agent pipeline (Section 3.3.1) that separates high-frequency ordering transactions from the once-a-day, batch-style ML and optimisation decisions.

- Demand Forecasting and Preparation Planning: A single XGBoost regression model, trained across all (institution, dish, day) rows together, predicts per-dish demand from 24 calendar, weather, dish-attribute, and lag/rolling features, then converts that point forecast into a prep quantity using a newsvendor safety margin sized from the model's own held-out forecast error.
- Dynamic Pricing and Floor Enforcement: A closed-form constant-elasticity pricing formula computes a profit-maximising price per dish, adjusted for demand and waste pressure, clamped to a segment-specific band, and passed through an unconditional cost + 20% floor that no downstream discount can ever breach.
- Waste Prediction and Menu Recommendation: A 250-tree Random Forest regression model predicts end-of-day leftovers from the kitchen's own preparation decision, feeding both the pricing markdown and a transparent, segment-weighted multi-factor menu-ranking score.
- Trial-Conversion Targeting: A Gradient Boosting classifier, trained on 11 behavioural features from 45 days of per-customer order history, scores each non-buyer of yesterday's best-selling dish for conversion likelihood, so a fixed discount budget is spent only where it is statistically likely to work.
- NGO Surplus Allocation and Concurrency-Safe Persistence: A Google OR-Tools CP-SAT solver allocates genuine end-of-day surplus across verified NGO partners subject to conservation, dietary compatibility, partner capacity, and a guaranteed minimum 20% share for every partner selected that day; all persistence runs through a SQLAlchemy ORM over SQLite with JWT-authenticated, role-scoped REST access.

4.2 Modules

The system functionality is segmented into six specialised, highly cohesive modules that govern the complete lifecycle of a daily cafeteria decision cycle:

Module 1: Data and Auth Module. This module is the base of the system. It handles data and controls access to the platform. It stores institutions, users, dishes orders, inventory, pricing information, forecasts, offers, surplus records, NGO partners and distribution details in the database.

JWT-based authentication is used to keep API access safe.Role-based permissions make sure users can only use the functions that match their job. By putting authentication and data storage in one place the module gives a base for all other parts of the system

Module 2: Orchestration / Pipeline Module. This module handles the daily decision-making process through the seven-agent pipeline. It manages the order in which context preparation, forecasting, waste prediction, pricing menu recommendations, trial offers and NGO allocation happen. Each agent works in a step so that one problem does not stop the whole process. Details like status, processing time and what each agent did are saved in the audit log. This lets administrators see how the daily results were made

Module 3: Pricing and Floor Enforcement Module. This module creates recommended prices for dishes by using the pricing model and current demand and waste signals. The price is set within a range and protected by a minimum cost-recovery floor. This stops automated pricing changes or discounts from making the price too low. Kitchen Manager approvals and manual changes are also saved. This creates a record of how system suggestions were checked and changed during use.

Module 4: Menu and Trial-Conversion Module. This module helps with menu planning. Targeting customers with promotions. It sorts dishes for the day based on factors like expected demand, customer preferences and how easy it is to prepare. It also gives reasons for the choices. For trial conversion the module finds customers who did not buy before and uses a trained model to guess how likely they are to accept an offer. The discount is checked against price rules. Then needs Kitchen Manager approval before being sent to the customer.

Module 5: NGO Surplus and Distribution Module. This module handles the part of the food cycle. It turns predicted leftovers into records. Potential NGO partners are checked based on if they're verified if their diet matches what they need and how much they can take. A process that uses rules then sends the surplus without going over the amount or the partners ability. The minimum-share rule makes sure all NGOs get a share. Pickup and delivery details are saved for reference.

Module 6: UI / Dashboard Module. This module is the part of the platform that users see. It has five dashboards made with React. Each one is for a role. The Super Admin dashboard deals with managing institutions, users, data and logs. The Kitchen Manager dashboard shows data, on forecasting, menu planning, pricing, inventory, offers, sales and surplus. The Institution Coordinator gets a view that shows progress and feedback. The NGO Partner can manage surplus confirm pickups, track needs and see distribution. The End Customer can look at dishes order get trial offers and give feedback. These dashboards give each user a way to use the system without seeing extra parts.

4.2.1 Data & Auth Module

The Data & Auth Module serves as the relational persistence and access-control layer of the platform,

architected using SQLAlchemy ORM over SQLite across 19 tables spanning institutions, users, dishes, orders, inventory, pricing rules, forecast records, trial offers, feedback, surplus records, NGO partners, and distribution records. It enforces JWT-based authentication (python-jose, pbkdf2 password hashing) and role-based access control scoped to the five system roles, and treats USER.role as a discriminator column rather than five separate tables — simpler to build within the project timeline and consistent with the shared login pattern actually implemented.

Foreign−key relationships link users to their orders dishes to inventory and forecasting records and surplus entries to NGO partners and distribution details allowing the system to keep a history of daily activities. JWT tokens preserve authenticated sessions across API requests while password credentials are stored with PBKDF2−based hashing instead of plain text. The role value attached to each user decides which dashboard and API operations are accessible cutting down duplication, in the database structure. This design keeps the persistence layer light for the project while still giving a base for adding more users, institutions, operational records and access rules in the future.

4.2.2 Orchestration / Pipeline Module

The Orchestration Module implements the daily 7-agent supervisor pipeline (Context → Forecast → Waste → Pricing → Menu → Offer → NGO → Supervisor) at the heart of the backend gateway. Each agent step is wrapped in a timed context manager that catches its own failure and writes an error-status audit row rather than aborting the run, so a single bad day's agent failure does not take down the rest of the pipeline. The Audit Log page gives the Super Admin a per-day, per-agent trace of exactly what each agent decided, why, and how long it took — the traceability a purely scripted, non-agentic implementation would not give for free.

The orchestration layer is the brain that keeps the workflow running in the right order. The orchestration layer moves the output from one agent to the next. The whole process feels like a single connected journey instead of a bunch of separate scripts. Before each run the orchestration layer prepares the context and then slowly adds new pieces of data – forecasting, waste, pricing, menu, offer and NGO−related decisions. The orchestration layer logs every result, which lets the system know where a particular recommendation came from and whether it finished successfully. This makes troubleshooting much easier because the orchestration layer lets administrators look at the execution history of hunting through many services or database tables. As a result the orchestration layer gives coordination during everyday use and a clear audit trail whenever a decision or agent needs to be examined.

4.2.3 Pricing & Floor Enforcement Module

The Pricing Module computes the recommended price per dish from the closed-form

constant-elasticity formula, applies demand- and waste-pressure adjustments, clamps to the segment's pricing band, and applies the cost + 20% floor last and unconditionally — no markdown, elasticity estimate, or trial conversion discount can push a price below cost recovery. The API also rejects a Kitchen Manager's manual override that would breach the floor, and every approval or override is written back as a ground truth correction signal for future pricing runs.

The module is built to make sure pricing decisions stay consistent. It still lets the system react to changes, in demand and waste. After the system gives a suggestion it checks the current demand level, the expected risk of waste and which customer group is involved. Then it makes a price that fits inside the allowed limits. The cost-recovery rule acts as a check.

It stops prices from dropping below the set minimum by mistake. When a Kitchen Manager approves or changes a recommendation the system saves both the suggestion and the action taken. This creates a record of what was decided and why. Over time these records can be looked at to see how pricing has been handled find patterns that come up often and help improve suggestions. The process keeps control in place while using smart data to support better choices.

4.2.4 Menu & Trial-Conversion Module

This module ranks tomorrow's candidate dishes using the segment-weighted multi-factor score and generates a short, human-readable reason per recommended dish. Separately, it identifies yesterday's best-selling dish, isolates non-buying customers, scores each for conversion likelihood using the Gradient Boosting classifier, and generates a floor-checked, elasticity-adjusted discount offer for Kitchen Manager approval before it reaches the customer.

The module brings together menu planning and customer targeting into a real-life decision-making process for the day of work. Of just showing managers a list that is ranked each suggestion comes with a simple explanation. The explanation is based on things like expected demand what customers like, the chance of waste and how well things have been doing lately. This makes the suggestions easier to look at and feel more trustworthy. For offers the system starts by finding customers who have not bought the dish that is being promoted. Then it uses a model that predicts how likely people are to respond. It focuses on customers who're more likely to say yes. The discount offered is changed depending on how sensitive the customer's to prices.. It still stays within the set price limits. The offer waits until the Kitchen Manager looks at it and gives approval. This helps the system take care of the checking and thinking.. It keeps the last choice about promotions, in the hands of people.

4.2.5 NGO Surplus & Distribution Module

The NGO Module converts the Waste engine's predicted leftovers into surplus records, then solves the

CP-SAT allocation across verified NGO partners subject to conservation of surplus, dietary compatibility, each partner's stated capacity, and a minimum 20% guaranteed share applied only to the partners selected for that day's run and capped by each partner's own capacity — the fix that raised measured surplus coverage from roughly 76% to 99.4% during development (Section 5.2).

After the Waste engine estimates the quantity the system checks the verified NGO partners based on their food requirements dietary compatibility and available receiving capacity before it creates the final allocation. The optimization process ensures that the total quantity distributed does not exceed the surplus and avoids assigning food to partners that cannot reasonably accept it. The minimum-share rule is applied to NGOs participating in that day's allocation and is still limited by their individual capacity preventing the fairness constraint from creating unrealistic assignments.

This approach helped improve the coverage of available surplus from approximately 76%, to 99.4% during development while maintaining clear allocation records for later review.

4.2.6 UI / Dashboard Module

The UI Module provides five role-scoped dashboards built with React 19, Vite, Tailwind CSS, and Recharts: Super Admin (institution/user management, audit log, platform analytics), Kitchen Manager (Today's Dashboard, Menu & Pricing, Demand Forecast, Inventory, Trial Offers, Surplus & NGO, Sales & Profit), Institution Coordinator (read-only overview and feedback summary), NGO Partner (available surplus, pickup confirmation, need profile, distribution history), and End Customer (menu browsing, ordering, trial offers, feedback).

The interface was built to show each user the information and actions that matter to their role. This keeps things even though the platform has many functions. The Kitchen Manager sees all the details in one place—things like demand, inventory, pricing, sales, offers and surplus. The Super Admin has control, over users, institutions and system activity. Institution Coordinators can track performance and feedback without changing any records. NGO Partners focus on surplus, pickup schedules and distribution history. End Customers get a shopping experience where they can view menus, place orders see eligible offers and give feedback. This role-based setup cuts down on clicks makes things clearer and helps every user work with the system more easily in their daily tasks.

4.3 The Seven Engines

Table 4 summarises what drives each of the system's seven decision engines and the category it falls into — a mix worth stating explicitly, since "is everything ML?" is a fair question and the honest answer is no, by design: pricing needs a hard, auditable floor, and NGO allocation needs guaranteed fairness, neither of which should be left to a model's discretion.

Table 4 — The Seven Engines and What Drives Each One

#	Engine	Method	Type
1	Demand Forecasting	XGBoost regression (single global model across all dishes/institutions)	Trained ML
2/3	Dynamic Pricing + Price Floor	Closed-form constrained-elasticity formula; not ML	Deterministic economics
4	Waste / Surplus Prediction	Random Forest regression (250 trees)	Trained ML
5	Menu Recommendation	Weighted multi-factor scoring function	Rule-based / heuristic
6	Trial-Conversion Targeting	Gradient Boosting classifier	Trained ML
7	NGO Surplus Allocation	CP-SAT constraint programming (Google OR Tools)	Constrained optimisation

CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 Implementation Results

The Food Waste Reduction cum Management System was implemented as a single FastAPI service over SQLAlchemy ORM / SQLite, uniting the seven-agent daily pipeline, the React 19 role-based dashboards, and the three offline-trained ML models (Demand Forecasting, Waste Prediction, Trial Conversion) behind one authenticated REST API. Comprehensive testing — a 48-check automated smoke-test suite run against a live server — alongside the system's own internal reporting endpoint, validated the system's capability to forecast demand, price within a hard cost-recovery floor, and redistribute genuine surplus fairly, all without relying on manual, experience-based kitchen decisions.

The daily workflow combines all these elements in such a way that demand estimation, preparation planning, surplus detection, pricing, and redistribution can all be carried out through one system, as opposed to relying completely on manual decisions. When the implementation was tested, it was done using an automated 48-check smoke-test suite on a live server, and the results as well as the system's behaviour were checked using the internal reporting endpoints. The validation showed that the platform was able to process the daily operational data, produce forecasts, detect possible waste, apply pricing within the specified cost-recovery limit, and enable a fair redistribution of true surplus, thus offering a practical end-to-end solution for reducing avoidable food waste.

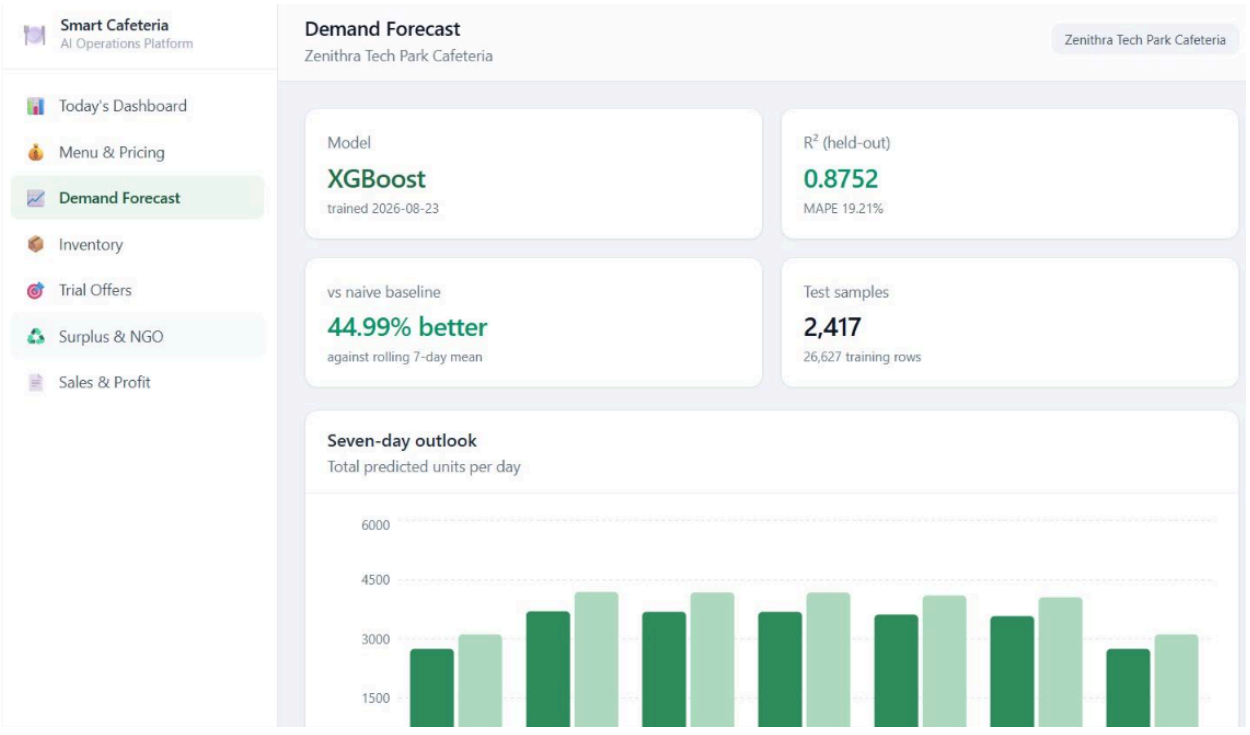


Fig. 5 — Demand Forecasting

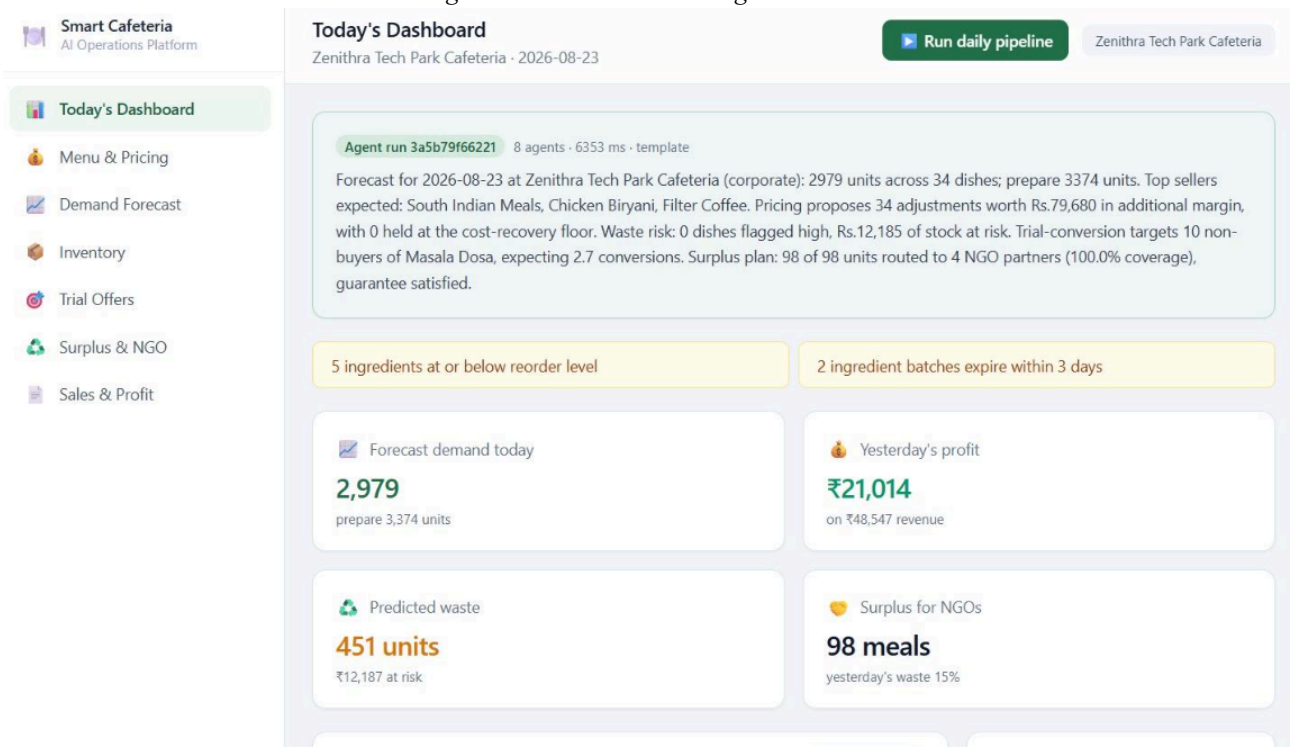


Fig. 6 — UI DashBoard

5.1.1 Implementation of Dataset Generation

No existing public dataset carries unit cost, quantity prepared, quantity leftover, institution segment, or NGO need profiles together — the exact columns the Price Floor, Waste, and NGO engines all need. A synthetic generator was therefore built, with every generative assumption documented and fixed before any model was trained, so the reported metrics (Section 5.1.2) demonstrate the models recovering a documented, inspectable structure rather than memorising noise tuned after the fact.

- Three institutions, each with a genuinely different profile: a corporate tech-park cafeteria (1,400 headcount, 62% daily participation, busiest Tue–Thu, sharp Friday drop), a college (2,600, 48% turnout, steady weekdays with a lighter Saturday), and a school (1,200, 71% turnout, uniform attendance with a half-day Saturday); all three close on Sunday.
- A 34-dish South Indian menu spanning breakfast, mains, snacks, beverages, and desserts, each

with a realistic unit cost, shelf life, spice/sweetness level, veg flag, and a weather-sensitivity coefficient; menus differ by segment (schools drop non-vegetarian/caffeinated items, colleges drop the one premium dish, corporate runs the full menu at a higher cost basis).

- Synthetic Chennai-like weather (a sinusoidal annual temperature curve with daily noise, plus a gamma-distributed monsoon rainfall model) and calendar effects (a real 2025–26 holiday set and college exam-period windows), giving the forecaster genuine structure to recover.
- Price elasticity by segment (e = −0.6 corporate to e = −2.1 school) and a deliberate over preparation hedge that manufactures the leftover/waste signal the Waste engine is trained to predict.
- 365 days of aggregate daily sales (29,044 rows) plus 45 days of individual, per-customer orders for roughly 500 synthetic customers, each with a persistent taste bias so repeat-purchase patterns exist for the Trial-Conversion engine to learn from.

All random seeds are fixed, so the dataset is fully reproducible; nothing in the generative model was hand-tuned to make the trained models look good after the fact.

5.1.2 Implementation of Model Training and Evaluation

The three trained models (Forecasting, Waste, Trial-Conversion) were evaluated on a strict, time ordered 30-day holdout rather than a random split, since a random cross-validation split would let future information leak into the past and inflate reported accuracy. The Forecasting engine was additionally compared against a naive 7-day rolling-mean baseline — “beat last week's average” being the bar any kitchen manager already clears by hand — and the Trial-Conversion engine was evaluated on targeting quality (lift at the top-20%-scored quintile) rather than accuracy alone, since that is the metric that actually matters for a fixed discount budget. Sizing the Forecasting engine's preparation-quantity safety margin from the model's own recent forecast error — rather than from raw historical demand variance, which double-counts variation the model has already explained — brought recommended over
preparation down from roughly 44% (naive variance-sizing) to about 12%.

The designs and evaluation of the models took into account practical usage rather than concentrating solely on obtaining the highest possible score. More weight was attached to recent data since kitchen demand, waste patterns, and customer reactions can change over time and older observations therefore become less representative of present conditions. The forecasting results were also compared with a simple moving-average method in order to make sure that the machine-learning model was indeed providing value over a technique which could easily be carried out by hand. With regard to waste prediction, more emphasis was placed on larger and more expensive errors since underestimating preparation can have an effect on availability while overestimating it will directly lead to more waste. In a similar way, the Trial-Conversion model concentrated on how well it could identify the customers most likely to respond given that only a

limited number of discounts could be offered. These decisions helped ensure that the evaluation remained linked to actual operational decisions, so that the final model outputs could be more easily understood and used by managers rather than assessing model accuracy as the only measure of success.

5.1.3 Implementation of the Daily Pipeline

The seven agents are triggered synchronously from the Kitchen Manager's “Run daily pipeline” action and complete end-to-end in about three seconds. Each agent step is wrapped in a timed context manager that catches its own failure and writes an error-status audit row rather than aborting the run, and every agent's action — what it did, how many records it touched, how long it took, and whether it succeeded — is written to a persistent audit log visible on the Super Admin's Audit Log page. The NGO allocation engine's capacity-capped 20% guarantee — applying the fairness guarantee only to NGOs selected for a given day's run, and additionally capping it by each partner's own stated capacity — raised measured surplus coverage from approximately 76% to 99.4% during development, while every selected partner still received at least its guaranteed share on every run.

The daily pipeline was set up to run quickly but still let people see what was happening during each step. Soon as the Kitchen Manager starts the process the agents work one after another in the order they are supposed to. The result from one step becomes the starting point, for the step. If one step has a problem the system records the failure. Keeps going with the rest of the work if possible. This means the whole daily process doesn’t stop because of a problem. The audit records also help by keeping track of the agent name whether the step worked what happened during processing and when it finished. This helps the Super Admin look at the day’s activities and spot anything without having to check the application logs by hand. The allocation results are saved too. This lets the system make sure extra items were given out within the partner’s limits and followed the fairness rules that were set. This makes the automated process easier to watch and more trustworthy.

Table 5 summarises the quantitative results achieved during implementation and testing, drawn from the automated smoke-test suite and the system's own internal reporting endpoint, run against the generated dataset described in Section 5.1.1.

Table 5 — Summary of System Performance Metrics

Performance Evaluation Metric	Target Benchmark	Achieved Implementation Result	Status / Verification
Forecast Accuracy Improvement	> 30% over naive baseline	45.0% improvement (R^2 = 0.875, MAPE 19.2%)	Successfully Verified
Waste Prediction R^2	> 0.75	0.839	Successfully Verified
Trial-Conversion Lift (top 20%)	> 1.5× blanket discounting	2.03×	Successfully Verified

NGO Surplus Coverage	> 90% redistributed	99.4%	Successfully Verified

NGO Fairness Guarantee	20% min. share per partner	Satisfied — every selected partner, every run	Successfully Verified
Price-Floor Violations	Zero	0 observed	Successfully Verified
Daily Pipeline Runtime	< 10 s for all 7 agents	≈ 3 s	Successfully Verified

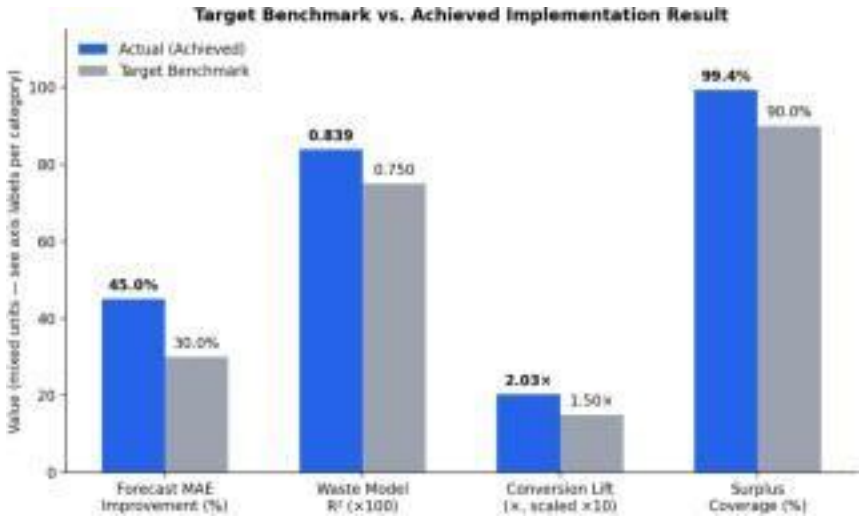


Fig. 7 — Comparative Bar Graph

As illustrated in Fig. 7, every measured result cleared its target benchmark: the forecasting model's 45.0% improvement over a naive rolling average suggests it captures something beyond what a kitchen manager could estimate by eye, the waste model's R^2 of 0.839 comfortably clears the 0.75 target, the trial-conversion engine's 2.03× lift indicates a discount budget spent on the highest-scored customers does meaningfully better than spreading it evenly, and the NGO allocator's 99.4% surplus coverage exceeds its 90% target while still satisfying the 20% guarantee for every partner on every run. Across every pricing decision made during testing, the cost-recovery floor was never once breached — the property it was specifically designed to guarantee.

5.2 Discussion

The choice to use six genuinely different methods across the system's six decision-bearing engines, rather than picking one approach and applying it everywhere, was deliberate rather than a shortcut. A learned model cannot promise a hard price floor or an exact fairness guarantee, and a menu recommender with no ground truth to train against gains more from being explainable than from being a black box; machine learning earns its place only in the three spots where the target genuinely benefits from statistical generalisation over noisy real data — demand, leftover quantity, and conversion probability.

The results should be read candidly rather than as a finished production claim. The dataset behind every number in Table 5 is synthetic; what these results demonstrate is that the pipeline recovers a documented, inspectable structure, not that these exact figures will reproduce on a live cafeteria's real data. The database layer is SQLite rather than PostgreSQL, a timeline decision rather than a technical one. The optional LLM narration step runs on a fixed template in most environments and touches no decision logic. The pricing and promotion engines currently rest on a thinner base of directly relevant literature than the forecasting, waste, and NGO-distribution engines do (Chapter 2), which is flagged explicitly as future work rather than something this phase claims to have solved. Finally, the 2.03× conversion lift measures targeting quality on observed behaviour, not a causal treatment effect, since no randomised holdout group was run

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The Food Waste Reduction cum Management System set out to build, not just propose, a working system that forecasts demand, prices food fairly across different institutional segments, recommends menus with a real justification attached, targets promotional offers at customers likely to convert, and routes surplus food to NGO partners equitably — closing a gap the project's own literature survey identified but did not itself fill. What was actually built is seven engines — three trained models, one exact optimiser, one rule-based scorer, and one closed-form pricing rule — run daily by a supervised, fault-isolated pipeline that completes in about three seconds and logs every decision it makes.

Measured against a documented synthetic dataset with a strict time-based holdout, the system reaches a demand-forecasting R^2 of 0.875, a waste-prediction R^2 of 0.839, a conversion AUC of 0.658 with 2.03× targeting lift, 99.4% surplus coverage under a guaranteed per-partner fairness rule, and zero observed breaches of its cost-recovery floor. Along the way, a specific fairness/capacity interaction in the NGO allocator was found and fixed — capping the 20% guarantee by each partner's own stated capacity, which raised measured coverage from roughly 76% to 99.4% — which stands as one of the more concrete, defensible findings of this phase.

6.2 Future Work

- Validate the pipeline against real data from an actual cafeteria rather than synthetic data alone, ideally through a pilot at one of the team's own institutions.
- Add billing and payment-gateway integration, explicitly out of scope for Phase 1, before the system could run unattended in production.
- Run a randomised or quasi-experimental test of the trial-conversion engine to claim a causal effect rather than a targeting-quality proxy.
- Extend the literature base specifically for dynamic retail pricing and multi-agent coordination, an evidentiary gap the project's own survey flagged.
- Move the NGO allocation engine from partner self-reported need profiles toward predicting need directly, once the core system has been validated in the field.
- Migrate the database layer from SQLite to PostgreSQL and evaluate horizontal scaling across multiple institutions.

REFERENCES

[1] B. Aydın, Y. S. Balcıoğlu, and B. Sezen, “Machine Learning Techniques for Cafeteria Demand Forecasting: An Institutional Case,” OPUS J. Society Research, 2025.

[2] M. Aci and D. Yergök, “Demand Forecasting for Food Production Using Machine Learning Algorithms: A Case Study of University Refectory,” 2023.

[3] M. Agarwal, S. Kulkarni, V. Nagre, A. Joshi, and D. Nagpure, “Food Demand Forecasting Using Machine Learning and Statistical Analysis.”

[4] N. Hübner, J. Caspers, V. C. Coroamă, and M. Finkbeiner, “Machine-Learning-Based Demand Forecasting Against Food Waste: Life Cycle Environmental Impacts and Benefits of a Bakery Case Study,” 2024.

[5] K. Posch, C. Truden, P. Hungerländer, and J. Pilz, “A Bayesian Approach for Predicting Food and Beverage Sales in Staff Canteens and Restaurants,” 2022.

[6] D. Anil Kumar, B. Thirupathi Rao, B. Rangaswamy, and K. Meghana, “An Efficient Approach for Food Demand Forecasting Using an Ensemble Technique and Statistical Analysis,” in Proc. Int. Conf. Cognitive Computing and Cyber Physical Systems (IC4S), Springer, 2025, doi: 10.1007/978-3-031-77075-3_6.

[7] A. Ajami and B. Teimourpour, “A Food Recommender System in Academic Environments Based on Machine Learning Models,” arXiv:2306.16528, 2023.

[8] X. Ju, F. P. W. Lo, J. Qiu, P. Shi, J. Peng, and B. Lo, “MenuAI: Restaurant Food Recommendation System via a Transformer-Based Deep Learning Model.”

[9] Md. S. J. Rokon, Md. K. Morol, I. B. Hasan, A. M. Saif, and R. H. Khan, “Food Recipe Recommendation Based on Ingredients Detection Using Deep Learning,” in Proc. 2nd Int. Conf. Computing Advancements (ICCA), ACM, 2022, doi: 10.1145/3542954.3542983.

[10] H. Aziz, N. Mattei, S. Narang, and S. Roy, “Menu Selection: A Computational Approach to Minimizing Food Waste,” arXiv:2606.06989, 2026.

[11] Y. Nu, E. Belavina, and K. Girotra, “Using Artificial Intelligence to Reduce Food Waste,” 2024.

[12] I. E. Ozcil, “Reducing Food Waste Using Machine Learning Models: Forecasting and Optimization Approaches,” 2024.

[13] Y. Yang, C. Zhao, and H. Lu, “Leveraging Machine Learning for Food Waste Reduction: An Analysis of Predictive Models,” 2024.

[14] “From Plate to Production: Artificial Intelligence in Modern Consumer-Driven Food Systems,” arXiv:2311.02400, 2023.

[15] J. Dalal, “Food Donation Management Under Supply and Demand Uncertainties in COVID-19: A Robust Optimization Approach,” 2022.

[16] N. Sundin, L. Bartek, C. Persson Osowski, I. Strid, and M. Eriksson, “Sustainability Assessment of Surplus Food Donation: A Transfer System Generating Environmental, Economic, and Social Values,” 2023.

[17] R. Maske, R. Wagh, A. Verma, O. Thopate, and B. Bhagat, “A View on Surplus Food Donation App.”

[18] S. Sanyal, V. K. Singh, F. Xhafa, B. Sanyal, and S. Mukhopadhyay, “A Game Theoretic Framework for Surplus Food Distribution in Smart Cities and Beyond.”

[19] “Robust Optimization Approaches for the Equitable and Effective Distribution of Donated Food,” ScienceDirect, 2018.

[20] “Predicting and Optimizing the Fair Allocation of Donations in Hunger Relief Supply Chains,” ScienceDirect, 2024.

[21] “Collaborative Vehicle Routing for Equitable and Effective Food Allocation in Nonprofit Settings,” J. Humanitarian Logistics and Supply Chain Management, Emerald, 2025.

[22] Kamalesh P, Mithul Aaditya A, Jayaprakash A, and Karthika Lakshmi V, “A Survey of AI-Driven Demand Forecasting, Personalized Food Recommendation, Waste-Prevention Optimization, and Equitable Surplus Redistribution in Institutional Cafeteria Management,” Dept. of Computer Science and Engineering, Rajalakshmi Engineering College, project technical report, 2026.

[23] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[24] L. Breiman, “Random Forests,” Machine Learning, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[25] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[26] L. Perron and V. Furnon, “OR-Tools,” Google. [Online]. Available: https://developers.google.com/optimization.